

Stochastic globular cluster simulations of binary black hole dynamical interactions

In this notebook we present an emulator for gravitational NBODY simulations of globular clusters, given a i) black hole population $N(m)$ and ii) initial conditions r_h, N_{stars} for the NBODY simulation. We make a comparison with state of the art globular cluster emulators by Kritos et al . <https://arxiv.org/pdf/2210.10055>

For reference, we will also compare with an NBODY simulation which we have at hand from DeVita et al. 2019

<https://arxiv.org/pdf/1903.07619>

Constants and functions

all in SI units

physical constants

```
In[1]: Mpcm = 106 × 3.0856775807 × 1016; (*Mpc in meters*)
pc = 3.0856775807 × 1016; (*pc in m*)
c = 299 792 458; (*speed of light, m/s*)
G = 6.67428 × 10-11; (*G in m3/kg/s2*)
Mpc = Mpcm / c; (*Gpc in seconds*)
yr = 3.15569 × 107; (*year in seconds*)
AU = 149 597 871 × 103; (*m*)
Rsun = 695 500 000; (*m*)
Msunkg = 1.98892 × 1030; (*Msun in kg*)
tHubble = 1.4 × 1010 yr;
thubble = 14 × 109 π 107;
Myr = π 107 × 106;
```

Cluster properties (can200k model)

```
In[13]: Rh = 4 pc; (*Half light radius in m*)
mav = 0.4;
M = 2 × 105 Msunkg mav; (*total mass in kg*)
Ra = Rh / 1.3; (*Plummer sphere*)
v = (G M / Ra)1/2;
vesc0 = 3 (G M / Rh)1/2;
```

Note that the black hole subcluster has a much smaller half mass radius. We take this directly from the NBODY simulation, which is not at time zero but at around 2 Gyr after the cluster has properly collapsed.

This comes from Equation 30 in Lee <https://arxiv.org/pdf/astro-ph/9409073>

It is

$$r_{h,BH} = 0.02 \frac{8}{\xi_{\min}} \left(\frac{MBH_{av}}{10} \right)^2 \frac{MBH}{1000 MBH_{av}} \frac{0.64}{m_{av}} \frac{10^6}{M_{tot}}$$

where

$$\xi_{\min} = \left(m_{bar}^3 M_{bar}^2 \left(\frac{27}{4} \left(1 + \frac{3 M_{bar}}{2 m_{bar}} \right) \left(1 + \frac{5}{2} M_{bar} \right)^{-3} \right) \right)^{1/3}$$

and

$$m_{bar} = \frac{M_{BH,av}}{m_{av}}$$

and

$$M_{bar} = \frac{M_{BH}}{M_{tot}}$$

From the can200k simulation, there are 109 black holes, $M_{BH,av} = 9.75$, $r_{h,BH} = 0.6$ pc, $m_{av} = 0.4$, $M_{tot} = 95,000$. Then

```
In[19]:= NBH1 = 109;
Rh = 4 pc;
MBHav = 9.75;
mav = 0.4;
Mtot = M / Msunkg;
mbar = MBHav / mav;
Mbar = MBHav 109 / (Mtot);
ξmin = (mbar^3 Mbar^2 (27/4 (1 + 3 Mbar / (2 mbar)) (1 + 5/2 Mbar)^-3))^(1/3);
rhBHLee = Rh / pc 0.04 4 / ξmin (MBHav / 10)^2 NBH1 / 1000 mav / Mtot
```

```
Out[27]=
0.497066
```

Note that these quantities are taken from the initial conditions . But we are using them to compute a quantity once the cluster is actually collapsed (which may be Gyr later, where the quantities such as mean mass .. total cluster mass.. change slightly). So we should be careful.

```
In[28]:=
```

Loading the actual mass distribution of black holes in can200k

```
In[29]:= SetDirectory["/Users/lucym/Documents/2026/black-hole-triples"]
Out[29]=
/Users/lucym/Documents/2026/black-hole-triples

In[30]:= Filename = "can200k_masses.csv";
can200kBHmass = Select[NumericQ][Import[Filename, {"Data", All, 1}]];
```

functions

Energy distribution from Stone and Leigh 2019 <https://arxiv.org/abs/1909.05272>

```
In[32]:= dista[x_] = ProbabilityDistribution[ $\frac{5}{2} x^{3/2}$ , {x, 0, 1}];
```

Eccentricities (from Stone and Leigh, around middle of page 5)

```
In[33]:= diste[x_] = ProbabilityDistribution[ $\left(\frac{4+\pi}{8}\right)^{-1} \frac{x(1+x)}{2(1-x^2)^{1/2}}$ , {x, 0, 1}];
```

```
distetherm[x_] = ProbabilityDistribution[2 x, {x, 0, 1}];
```

We take the probability that the interloper m3 component is NOT swapped out from Ginat and Perets 2019:

```
In[35]:= Pswap[m11_, m22_, m33_] = 
$$\frac{(m11^4 m22^4)}{\left((m11 + m22)^{5/2} \left(m11^4 \left(\frac{m33^4}{(m33+m11)^{5/2}} + \frac{m22^4}{(m11+m22)^{5/2}}\right) + \frac{m33^4 m22^4}{(m33+m22)^{5/2}}\right)\right)}$$
;
```

The hardening factor if given by equation 11 in Kritos + 2024. Note equal mass case is 0.78.

Setting up cluster population

For the mass distribution, we can roughly reproduce the can200k cluster BH population with the following

```
In[36]:= NBHi = 140;
incluster = Table[1, {i, NBHi}];
mtemp = RandomVariate[ExponentialDistribution[0.7], NBHi] + 10;
mtemp = RandomVariate[GammaDistribution[8, 1.5], NBHi];
m = Pick[mtemp, 8 < # < 40 & /@ mtemp];
NBH = m // Length
```

Out[41]=

116

```
In[42]:= Mean[mtemp]
```

Out[42]=

12.1767

Pairing up black holes in binaries

Now we will pair up the black holes, given a binary fraction of 10 percent

```
In[43]:= fbin = 0.1;
Nbin = fbin NBH // Round
```

Out[44]=

12

We make sure that the majority (f1 = 80%?) of BH in binaries are the most massive . But not all .

```
In[45]:= f1 = 0.8;
```

```
In[46]:= mtemp = ReverseSort[m];
```

```
In[47]:= min3 = mtemp[[;; Round[f1 2 Nbin]]];
```

To reorder, we delete the BH in binaries from mtemp

```
In[48]:= mtemp[[;; Round[f1 2 Nbin]]] = -1;
```

```
In[49]:= mtemp2 = Select[mtemp, # > 0 &];
```

mnew is our jumbled binary population, dominated by most massive

```
In[50]:= mnew = Join[min3, RandomSample[mtemp2[[ ]]]];
```

```
In[51]:= minbinary0 = mnew[[;; 2 Nbin];
```

```
minterlopers0 = mnew[[2 Nbin + 1 ;;];
```

Now pair them up randomly

```
In[53]:= {testArrayOdd, testArrayEven} = Partition[minbinary0, 2, 2, 1, {}] ~Flatten~ {2};
```

```
In[54]:= Clear@groupByPositions
```

```
groupByPositions[list_, pred_] :=
```

```
With[{trueList = minbinary0[[Select[Range@Length@list, pred]]],
```

```
{trueList, Complement[list, trueList]}}
```

```
{testArrayOdd, testArrayEven} = groupByPositions[minbinary0, OddQ];
```

We arrange the ones in binaries so that more massive is at an odd index.

```
In[57]:= diffarray = testArrayEven - testArrayOdd;
```

```
In[58]:= m1big = Join[Pick[testArrayEven, # > 0 & /@diffarray],
```

```
Pick[testArrayOdd, # < 0 & /@diffarray] ];
```

```
In[59]:= m2small = Join[Pick[testArrayEven, # < 0 & /@diffarray],
```

```
Pick[testArrayOdd, # > 0 & /@diffarray] ];
```

```
In[60]:= mbin = m1big + m2small
```

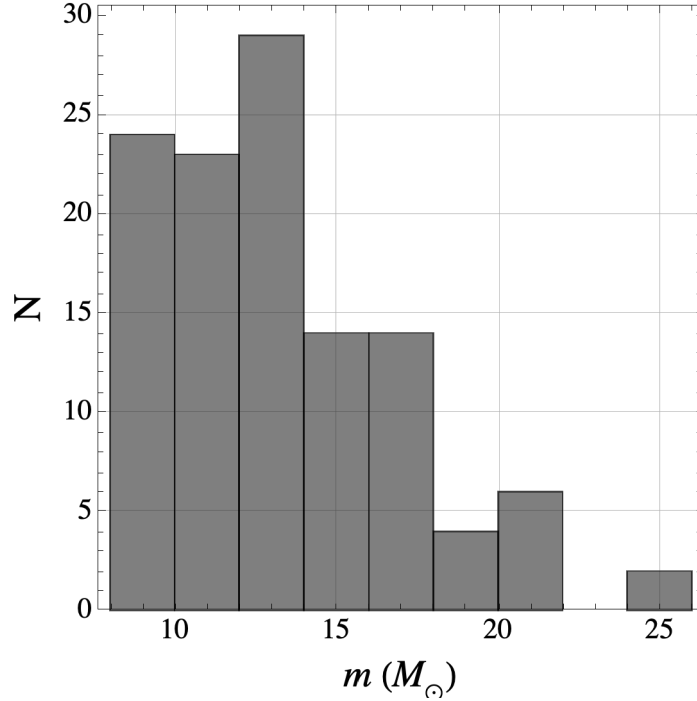
```
Out[60]=
```

```
{28.0995, 31.5869, 33.934, 37.916, 38.9775, 41.9123,
 42.0113, 39.2316, 38.4239, 37.7216, 33.2005, 28.1018}
```

Here we plot the black hole mass distribution

```
In[61]:= Histogram[mnew, ChartStyle → {Directive[Black, Opacity[.5]]}, AspectRatio → 1,
  Frame → True, FrameLabel → {Style["m (M⊙)", 20, Black], Style["N", 20, Black]},
  LabelStyle → {(FontFamily → "Times"), Black}, GridLines → Automatic,
  FrameTicksStyle → Directive[FontSize → 15, Black],
  ScalingFunctions → {None, None}, BaseStyle → {FontSize → 20, Black}]
```

Out[61]=



```
In[62]:= mnew // Length
```

Out[62]=

116

Print related quantities to make consistent $r_{h,BH}$

```
In[63]:= Mean[mnew]
```

Out[63]=

13.3439

```
In[64]:= Mean[mnew] × Length[mnew]
```

Out[64]=

1547.89

```
In[65]:= NBH1 MBHav
```

Out[65]=

1062.75

initial binary separations and eccentricities

For the initial separations, we draw from a log Uniform distribution with a maximum of the hard - soft boundary like Morscher + 2015

```
In[66]:= ahard = G (m1big + m2small) Msunkg / v2;
```

```
In[67]:= amin = Table[50 (Rsun + Rsun), {i, 1, Length[ahard]}];
```

Initial eccentricities drawn from a thermal distribution.

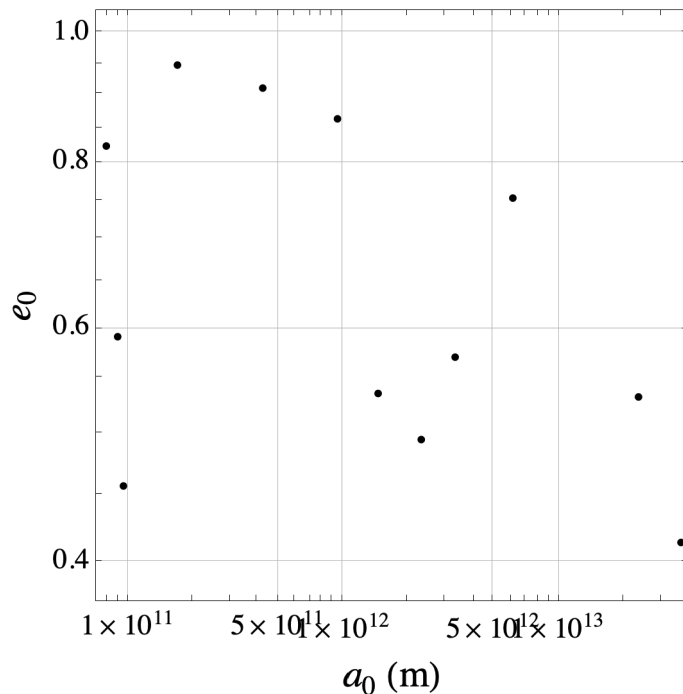
```
In[68]:= earrbin = Sqrt[RandomReal[{0, 1}, Nbin]]];

In[69]:= aarrbinlog = Table[RandomVariate[
    UniformDistribution[{Log[amin][[i]], Log[ahard][[i]]}], {i, 1, Length[ahard]}];

In[70]:= aarrbin2 = Exp[aarrbinlog];
aedata = Transpose[{aarrbin2, earrbin}];

In[72]:= ListLogLogPlot[aedata, AspectRatio → 1, Frame → True,
    FrameLabel → {Style[" $a_0$  (m)", 20, Black], Style[" $e_0$ ", 20, Black]},
    LabelStyle → {(FontFamily → "Times"), Black}, GridLines → Automatic,
    FrameTicksStyle → Directive[FontSize → 15, Black],
    ScalingFunctions → {None, None}, BaseStyle → {FontSize → 20, Black}, PlotRange →
    {{0.9 Min[aarrbin2], 1.1 Max[aarrbin2]}, {0.9 Min[earrbin], 1.1 Max[earrbin]}}]
```

Out[72]=



Stochastic cluster simulation: super thermal eccentricity sampling, statistical hardening

We create empty lists for the number of merged binaries, and ejected binaries. We also make separate lists for the masses of merged and ejected binaries.

```

In[73]:= nmsto = {};
         nesto = {};
         neisto = {};
         m1msto = {};
         m2msto = {};
         m1esto = {};
         m2esto = {};
         aejsto = {}
         eejsto = {}
         tejesto = {};
         tejmergesto = {};
         aejmergesto = {};
         eejmergesto = {};
         timemerge = {};
         nmergej = {};
         Nsim = 1000;

```

Out[80]=

```
{ }
```

Out[81]=

```
{ }
```

Note: Flag 15 for the eccentricity: we draw from a super thermal distribution. Something optional to be confirmed(?) by rebound is any correlation between the post-interaction e and a .

Also note that currently, we don't change the black hole sub cluster half mass radius rh_{BH} according to ejections.

```

In[89]:= ( (*Flag 1: Initialization for a single cluster simulation*)

         Niter = 0;
         Label[3];
         merge = Table[1, {i, Nbin}];
         in = Table[1, {i, Nbin}];
         totali = 0;
         massinint2 = {};
         massinint3 = {};
         aafter = {};
         totalmerge = 0;
         totaleject = 0;
         totalejecti = 0;
         totalejectmerge = 0;
         mbinloop = mbin;
         m1loop = m1big;
         m2loop = m2small;
         aloop = aarrbin2;

```

```

eloop = earrbin;
mintloop = minterlopers0;
aej = Table[1, {i, Nbin}];
NBHi = 150;
incluster = Table[1, {i, NBHi}];
rhBH0 = 0.73 pc;
rhBH = rhBH0;
rhfac = 1;
Rh = rhBH rhfac;
Ranew = Rh / 1.3;
tnow = 0;
nmax = thubble;

(
Label[1];

(* Flag 2 Compute GW merger timescale for all binaries *)
tauGWloop = c5 (G3 m1loop m2loop (mbinloop) (Msunkg)3)-1

$$\frac{5 aloop^4}{256} (1 + 0.27 eloop^{10} + 0.33 eloop^{20}) (1 - eloop^2)^{7/2};$$

vnew = v (Ra / Ranew)1/2;

NBHnow = mintloop // Length;
Ntime = 200  $\left( \frac{NBHnow}{4 / 3 \pi Rh^3} \right)$ ;

(* Flag 3: calculate interaction time
for each binary. Depends on number density (fixed)*)
tauintarrlooptmpd = vnew / (104 / pc3) /  $\pi$  / aloop / G / (mbinloop * Msunkg);
tauintarrloop = tauintarrlooptmpd;

timeloc =
With[{m = Min[tauintarrlooptmpd]}, {m, Position[tauintarrlooptmpd, m]}];
timeind = timeloc[[2]][[1]][[1]];
timetot = timeloc[[1]];
(* Flag 4: choosing the binary with the smallest merger timescale*)
timelocGW = With[{m = Min[tauGWloop]}, {m, Position[tauGWloop, m]}];
timeindGW = timelocGW[[2]][[1]][[1]];
timetotGW = timelocGW[[1]];

(* Flag 5: if a merger happens (in timetotGW) before the next soonest
interaction (timetot). In this case, starts loops again at Flag 1*)
If[timetotGW < timetot,

```



```

{
  (*Flag 6: After a merger, product is moved to interloper list.,
  and its components are removed from all binary related lists*)
  mintiloop = Append[mintiloop, mbinloop[[timeindGW]],
  m1msto = Append[m1msto, m1loop[[timeind]]];
  m2msto = Append[m2msto, m2loop[[timeind]]];
  timemerge = Append[timemerge, tnow];
  m1loop[[timeind]] = -1,
  m2loop[[timeind]] = -1,
  mbinloop[[timeind]] = -1,
  aloop[[timeind]] = -1,
  eloop[[timeind]] = -1,
  m1loop = Select[m1loop, # > 0 &],
  m2loop = Select[m2loop, # > 0 &],
  mbinloop = Select[mbinloop, # > 0 &],
  aloop = Select[aloop, # > 0 &],
  eloop = Select[eloop, # > 0 &],
  tnow = tnow + timetotGW // N,
  totalmerge = totalmerge + 1,
  Goto[1]}}];

(* Flag 7: if all GW merger timescales
  are longer than the next interaction, there is a 2+
  1 exchange encounter. choose a random single BH in the interloper list*)
targetm = RandomVariate[
  UniformDistribution[{Min[minterlopers0], Max[minterlopers0]}]];

interloperloc = With[{m = Min[Abs[targetm - mintloop]]},
  {m, Position[Abs[targetm - mintloop], m]}];

randintloop = interloperloc[[2]][[1]][[1]];
mintiloop = mintloop[[randintloop]];

(* Flag 8: add to list of masses
  (binary, and interloper) involved in an encounter massinint2*)
massinint2 = Append[massinint2, m1loop[[timeind]];
massinint2 = Append[massinint2, m2loop[[timeind]];
massinint2 = Append[massinint2, mintiloop];
(*Flag 9: Calculate the binary energy before interaction*)
Epre = - 
$$\frac{G m1loop[[timeind]] \times m2loop[[timeind]] M_{\text{sun}} k^2}{2 aloop[[timeind]]};$$


(*Flag 10 :
  determine the remaining binary components according to Ginat and Perets*)
swapdraw = RandomVariate[UniformDistribution[{0, 1}]];
PswapTarget = Pswap[m1loop[[timeind]], m2loop[[timeind]], mintiloop] // N;

```

```

(*Flag 11 : determine hardening factor (between 0 and 1),
according to Stone and Leigh's energy distribution*)
afrac = RandomVariate[dista[x]];

(*Flag 12: calculate the post interaction a,
accounting for masses of swapped in/out components*)

(*Flag 12a: if there is a swap according to Ginat and Perets,
interloper swaps in for a component. First,
determine which component it replaces also according to Ginat and Perets *)
If[swapdraw > PswapTarget,
  (*this is the probability that m2 gets ejected over m1*)
  ejectedprob =
    RandomVariate[BinomialDistribution[1, Pswap[mintiloop, m1loop[[timeind]],
      m2loop[[timeind]] / (Pswap[mintiloop, m1loop[[timeind]], m2loop[[timeind]] +
        Pswap[mintiloop, m2loop[[timeind]], m1loop[[timeind]])]];

  (* Flag 12: m2 is ejected*)
  If[ejectedprob == 1,
    {abinpost = aloop[[timeind]] afrac  $\left(\frac{\text{mintloop}[[\text{randintloop}]]}{\text{m2loop}[[\text{timeind}]]}\right)$ ;
    m2tmp = m2loop[[timeind]];
    m2loop[[timeind]] = mintiloop;
    mintloop[[randintloop]] = m2tmp;
    mbinloop[[timeind]] = m1loop[[timeind]] + m2loop[[timeind]];}]];

  (* Flag 12: m1 is ejected *)
  If[ejectedprob == 0,
    {abinpost = aloop[[timeind]] afrac  $\left(\frac{\text{mintloop}[[\text{randintloop}]]}{\text{m1loop}[[\text{timeind}]]}\right)$ ;
    m2tmp = m1loop[[timeind]];
    m1loop[[timeind]] = mintiloop;
    mintloop[[randintloop]] = m2tmp;
    mbinloop[[timeind]] = m1loop[[timeind]] + m2loop[[timeind]];}]]];

(*Flag 12b: if there is no swap according to Ginat and Perets,
harden the binary*)
If[swapdraw < PswapTarget, {abinpost = aloop[[timeind]] afrac}];

(*Flag 13: add to list of masses
(binary, and interloper) involved in an encounter massinint3*)
massinint3 = Append[massinint3, m1loop[[timeind]];
massinint3 = Append[massinint3, m2loop[[timeind]];
massinint3 = Append[massinint3, mintloop[[randintloop]];

```

```

(*Flag 14: add to list of masses
  (binary, and interloper) involved in an encounter massinint3*)
aafter = Append[aafter, afrac];
alooop[[timeind]] = abinpost;
(*Flag 15: draw new eccentricity from the super thermal
  distribution. This relation was suggested in Quinlan(?))*)
elooop[[timeind]] = RandomVariate[distetherm[x]];

massesarr = Sort[{m1loop[[timeind]], m2loop[[timeind]], mintloop[[timeind]]}];
oldlist = {m1loop[[timeind]], m2loop[[timeind]], mintloop[[timeind]]};

(*Flag 16: calculate the ejection velocity of interloper. If it's larger
  than the cluster escape velocity, remove it from interloper list.*)

(*Jan 27: make sure it's consistent with the before and after*)
(* solve two equations. First momentum,
  second is energy in the three body COM frame. In energy expressions, must
  account for new mass arrangement. The RHS is the old arrangement, Epre.*)

Epost = 
$$\frac{G (m1loop[[timeind]] \times m2loop[[timeind]]) Msunkg^2}{2 abinpost};$$

vintafter =
Abs[{via} /. Solve[{(m1loop[[timeind]] + m2loop[[timeind]]) vba == mintloop[[timeind]]
  via, 
$$\frac{(m1loop[[timeind]] + m2loop[[timeind]]) Msunkg vba^2}{2} +$$


$$\frac{mintloop[[timeind]] Msunkg via^2}{2} -$$


$$\frac{G (m1loop[[timeind]] \times m2loop[[timeind]]) Msunkg^2}{2 abinpost} ==$$

  Epre}, {via, vba}][[1]][[1]];
vbinafter =
Abs[{vba} /. Solve[{(m1loop[[timeind]] + m2loop[[timeind]]) vba == mintloop[[timeind]]
  via, 
$$\frac{(m1loop[[timeind]] + m2loop[[timeind]]) Msunkg vba^2}{2} +$$


$$\frac{mintloop[[timeind]] Msunkg via^2}{2} -$$


$$\frac{G (m1loop[[timeind]] \times m2loop[[timeind]]) Msunkg^2}{2 abinpost} ==$$

  Epre}, {via, vba}][[1]][[1]];

veject = vintafter;
Ninter = mintloop // Length;
minttmp = mintloop[[timeind]];
If[veject > vesc0,

```

```

{
  mintloop[[timeind]] = -1,
  mintloop = Select[mintloop, # > 0 &]]];
totalejecti = totalejecti + 1;

(*Flag 17: Check if the post interaction binary is
  ejected. If binary velocity exceeds cluster escape velocity,
  remove ejected binary components from the binary related lists.*)

vkick = vbinafter;

If[vkick > vesc0,
{
  m1esto = Append[m1esto, m1loop[[timeind]]];
  m2esto = Append[m2esto, m1loop[[timeind]]];
  aejsto = Append[aejsto, aloop[[timeind]]];
  eejsto = Append[eejsto, eloop[[timeind]]];
  (*here, the interaction already happened. Calculate GW merger timescale*)
  tauGWeject =
    c5 (G3 m1loop[[timeind]] × m2loop[[timeind]] (mbinloop[[timeind]]) (Msunkg)3)-1
    
$$\frac{5 \text{ aloop}[[\text{timeind}]]^4}{256} (1 + 0.27 \text{ eloop}[[\text{timeind}]]^{10} + 0.33 \text{ eloop}[[\text{timeind}]]^{20})$$

    (1 - eloop[[timeind]]2)7/2,
  If[tnow + tauGWeject < 14 × 109 yr,
    totalejectmerge = totalejectmerge + 1;
    tejsto = Append[tejsto, tnow + timetot];
    tejmergesto = Append[tejmergesto, tauGWeject];
    aejmergesto = Append[aejmergesto, aloop[[timeind]]];
    eejmergesto = Append[eejmergesto, eloop[[timeind]]];],
  mbinloop[[timeind]] = -1,
  m1loop[[timeind]] = -1,
  m2loop[[timeind]] = -1,
  aloop[[timeind]] = -1,
  eloop[[timeind]] = -1,
  mbinloop = Select[mbinloop, # > 0 &],
  m1loop = Select[m1loop, # > 0 &],
  m2loop = Select[m2loop, # > 0 &],
  aloop = Select[aloop, # > 0 &],
  eloop = Select[eloop, # > 0 &],
  totaleject = totaleject + 1,
}]]];

```

(*Flag 18: advance in time according to this interaction, back to Flag 2.*)

```

tnow = tnow + timetot;

totali++;
If[tnow < thubble, Goto[1]];
)
;

Niter++;
nmsto = Append[nmsto, totalmerge];
nesto = Append[nesto, totaleject];
neisto = Append[neisto, totalejecti];
nmergej = Append[nmergej, totalejectmerge];

If[Niter < Nsim, Goto[3]]
)

```

- ... **Solve**: Solve was unable to solve the system with inexact coefficients. The answer was obtained by solving a corresponding exact system and numericizing the result.
- ... **Solve**: Solve was unable to solve the system with inexact coefficients. The answer was obtained by solving a corresponding exact system and numericizing the result.
- ... **Solve**: Solve was unable to solve the system with inexact coefficients. The answer was obtained by solving a corresponding exact system and numericizing the result.
- ... **General**: Further output of Solve::ratnz will be suppressed during this calculation.
- ... **Append**: Nonatomic expression expected at position 1 in Append[14.8759, 38.9249].
- ... **Append**: Append called with 3 arguments; 1 or 2 arguments are expected.
- ... **Append**: Append called with 4 arguments; 1 or 2 arguments are expected.
- ... **Append**: Append called with 5 arguments; 1 or 2 arguments are expected.
- ... **General**: Further output of Append::argt will be suppressed during this calculation.
- ... **Append**: Nonatomic expression expected at position 1 in Append[9.67359, 24.9279].
- ... **Append**: Nonatomic expression expected at position 1 in Append[12.2582, 36.4682].
- ... **General**: Further output of Append::normal will be suppressed during this calculation.
- ... **Part**: Part 1 of {} does not exist.
- ... **Part**: Part 1 of {} does not exist.
- ... **Part**: Part 1 of {} does not exist.

⋯ General: Further output of Part::partw will be suppressed during this calculation.

In[90]:=

In[91]:= **nmergej;**

In[92]:= **nmergej // Length**

Out[92]=

1000

In[93]:= **Mean[nmergej] // N**

Out[93]=

0.705

Mean number of mergers per simulation :

In[94]:= **Mean[nmsto] // N**

Out[94]=

1.503

Mean number of ejections per simulation :

In[95]:= **Mean[nesto] // N**

Out[95]=

5.161

Mean mass ratio for the mergers :

In[96]:= **m1msto / m2msto // Mean**

Out[96]=

1.3274

In[97]:= **Mean[neisto] // N**

Out[97]=

55.342

In[98]:= **aejsto / (1.496 × 10¹¹) // N;**

In[99]:= **Median[aejsto] / (1.496 × 10¹¹) // N**

Out[99]=

0.777187

In[100]:=

1 - Median[eejsto]

Out[100]=

0.292955

In[101]:=

1 - eejsto;

In[102]:=

timemerge;

In[103]:=

tejmergesto // Length

Out[103]=

705

Contour plot of a_{eject} and e_{eject} for binaries which merge out of the cluster : comparison with Kritos+'s Fig 10

```

In[104]:=
  eejmergesto // Length
Out[104]=
  705

In[105]:=
  Mean[1 - eejsto]
Out[105]=
  0.334703

In[106]:=
  Mean[aejsto / (1.496 × 1011)]
Out[106]=
  0.820013

In[107]:=
  data = Transpose[{Log10[1 - eejmergesto], Log10[aejmergesto / (1.496 × 1011)]}];
  (*Calculate a nonparametric density estimate*)
  d = SmoothKernelDistribution[data];

  (*Evaluate the estimated density function over a grid
    of points and sort by the density values from high to low*)
  pdf = Reverse[
    Sort[Flatten[Table[PDF[d, {x, y}], {x, -3, 3, 0.05}, {y, -3, 3, 0.05}]]]];

  (*Create a table of cumulative pdf values
    that correspond to the volumes of interest*)
  cdf = Accumulate[pdf] / Total[pdf];
  contours = pdf[[Flatten[
    Table[FirstPosition[cdf, p_ /; p ≥ alpha], {alpha, {0.68, 0.95, 0.99}}]]]];

  c = contours;

```

In[113]:=

```

cPad = PadLeft[c, Length@c + 1, 1];
cPad = PadRight[cPad, Length@cPad + 1, 0];
quantileBounds = Partition[cPad, 2, 1];
splitUp =
  Table[Select[data, Last[i] < PDF[d, #] ≤ First[i] &], {i, quantileBounds}];

sdh = SmoothDensityHistogram[data, PlotRange → {{-4, 1}, {-3, 2}},
  AspectRatio → 2 / 3, Frame → True, LabelStyle → {(FontFamily → "Times"), Black},
  FrameLabel → {Style["1- eeject"], Style["aeject (AU)"]},
  BaseStyle → {FontSize → 20}, GridLines → Automatic];
cp = ContourPlot[PDF[d, {x, y}], {x, -4, 1}, {y, -3, 2},
  Contours → contours, ContourShading → None, PlotRange → All];
nonEmptyContours = Select[splitUp, Length[#] > 0 &];

coloredByContourPoints =
  ListPlot[nonEmptyContours, PlotStyle → {Black, Green, Orange, Red}];
Show[{sdh, cp, coloredByContourPoints}, PlotRange → All];

```

In[122]:=

```

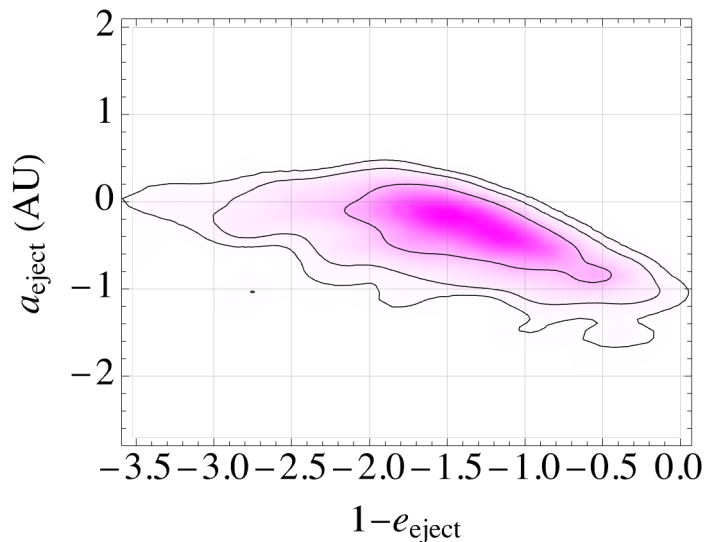
sdh3 = SmoothDensityHistogram[data, PlotRange → {{-3.52, 0}, {-2.7, 2}},
  AspectRatio → 3 / 4, Frame → True, LabelStyle → {(FontFamily → "Times"), Black},
  FrameLabel → {Style["1-eeject"], Style["aeject (AU)"]},
  BaseStyle → {FontSize → 20}, GridLines → Automatic,
  ColorFunction → (Blend[{White, Magenta}, #] &)];

```

In[123]:=

```
thermalcont = Show[{sdh3, cp}]
```

Out[123]=



CDF of mergers: comparison with Kritos+'s Fig 5

In[124]:=

```
timemerge // Length
```

Out[124]:=

```
1503
```

In[125]:=

```
mergetimes = Log10[timemerge / (106  $\pi$  107)];
```

In[126]:=

```
 $\mathcal{D}$  = EmpiricalDistribution[mergetimes]
```

Out[126]:=

```
DataDistribution[ Type: Empirical  
Data points: 1503]
```

In[127]:=

In[128]:=

```
mergercdf =  
DiscretePlot[5.3 CDF[ $\mathcal{D}$ , x], {x, 0, 5, .01}, ScalingFunctions → {None, "Log"},  
Frame → True, LabelStyle → {(FontFamily → "Times"), Black},  
FrameLabel → {Style["Log10(tmerge in Myr)", Style["N"]]},  
BaseStyle → {FontSize → 20}, GridLines → Automatic,  
ColorFunction → (Blend[{Magenta, Magenta}, #] &),  
PlotRange → {{0, 4.13}, {0.07, 5000}}, AspectRatio → 0.74]
```

Out[128]:=

